

The Principle of Open Systems: A Unified Analytical Framework for Interaction–Action–Reaction Dynamics

Macharia Bari

September 5, 2026

Abstract

Every closed subsystem seeks internal equilibrium; every open subsystem does something richer. When a subsystem exchanges matter, energy, or information with its environment, four fates are possible: it reaches an *internal equilibrium*, it settles into a *joint equilibrium* with its environment, it *dominates* the environment (or is dominated by it), or it *dissolves* into it. We show that these four fates—and the transitions between them—follow from a single recursive principle, **Interaction** $\rightarrow$ **Action** $\rightarrow$ **Reaction (IAR)**, written as four coupled ordinary differential equations with six dimensionless parameters. The framework admits closed-form solutions in controlled (closed) limits, realizes the Lotka–Volterra system and the Navier–Stokes equations as structural limits, produces a sharp resonance criterion $\alpha = 1/(1 + \theta)$, a stability threshold $\beta\gamma = \lambda$, and a critical Reynolds number $Re_c = 1/(\beta\gamma - \lambda)$. Initial calibration against COVID-19 case data, the S&P 500 index, and global temperature anomalies yields R^2 values of 0.92, 0.88, and 0.94 respectively. The framework is presented in closed form for controlled scenarios and in open form (an online, gradient-free, surrogate-driven engine, included as Appendix A) for streaming, spatial, noisy, and externally forced settings.

Contents

1	Introduction	1
2	Postulates	2
3	The Master Equations	2
4	Closed-Form Solutions	3
4.1	Interaction-ratio reduction and the resonance criterion	3
4.2	Lotka–Volterra embedding	4
4.3	Fixed points	5
5	The Four Fates	5
6	Stability, Intermittency, and Critical Scales	6
7	Embedding the Navier–Stokes Equations	8
8	Initial Empirical Validation	8
9	Open-Form Computation	9

1 Introduction

Open systems dominate the physical, biological, and social world. Stars exchange energy with interstellar space; cells exchange solutes with their extracellular fluid; firms exchange goods with markets; organisms exchange influence with their societies. Yet many of the best-known dynamical models are written for closed subsystems—isolated harmonic oscillators, closed-space kinetics, airtight balance sheets—with openness added afterward as a perturbation. The starting point of this work is the opposite ordering: *openness is primary*. A closed system is simply the special case of an open system in which the coupling with the environment has been turned off.

We propose a minimal, analytically tractable, and computationally open structure that remains well posed in three regimes at once: the *closed* regime, where exact solutions are available; the *semi-open* regime, where a small number of dimensionless parameters govern regime transitions (equilibrium, oscillation, dominance, dissolution); and the *fully open* regime, where no closed-form solution is assumed and everything is learned from streaming data.

The structure rests on three state variables and one interaction:

- X_C — the state of the **closed (controlled) subsystem** (the “Actor”).
- X_O — the state of the **open environment** (the “Other”).
- R — the accumulated **reaction potential** (memory, vorticity, immune response, contagion, sentiment).
- J — the **interaction** between Actor and Other.

The dynamics is the IAR loop:

Interaction $\rightarrow$ Action (changes to X_C) $\rightarrow$ Reaction (accumulation of R) $\rightarrow$ modified Interaction.

Section 2 states the three postulates. Section 3 gives the master equations. Section 4 derives the closed-form solutions. Section 5 establishes the four fates. Section 6 analyzes stability, oscillation, and intermittency. Section 7 shows that the Navier–Stokes equations are an operator-level realization of the IAR loop, and Section 4.2 that Lotka–Volterra dynamics is a rate-limited sub-manifold. Section 8 reports initial empirical validation. Section 10 discusses scope. Appendix A provides the open-form computational engine in full.

2 Postulates

Postulate 1 (Isolation). *In the absence of coupling, a subsystem relaxes to its own internal equilibrium, and the environment relaxes independently to its own.*

Postulate 2 (Coupling through interaction). *An open subsystem and its environment interact through a linear exchange law*

$$J = \beta (X_C - \theta X_O), \quad (1)$$

where $\beta > 0$ is the coupling (exchange) rate and $\theta > 0$ is the exchange ratio (relative weight of the environment in the interaction).

Postulate 3 (Reaction memory). *Changes in the subsystem accumulate in a reaction potential R that feeds back on the dynamics and decays with damping λ .*

Postulate 4 (Asymmetric influence). *The single parameter $\alpha \in [0, 1]$ apportions influence: a fraction $1 - \alpha$ of the interaction drives the subsystem X_C , and a fraction α of the reaction potential drives the environment X_O . $\alpha = 1/2$ is perfectly symmetric; $\alpha > 1/2$ means the reaction (and therefore the Actor) dominates the interface.*

3 The Master Equations

Under Postulates 1–4, the dynamics of an open system is

$$\dot{X}_C = (1 - \alpha) J + (\text{internal terms}), \quad (2)$$

$$\dot{X}_O = \alpha R + (\text{internal terms}), \quad (3)$$

$$\dot{R} = \gamma J - \lambda R - \eta X_O R, \quad (4)$$

$$J = \beta(X_C - \theta X_O).$$

The terms have the following meaning:

- γ : rate at which interaction is converted into accumulated reaction potential.
- λ : passive decay of the reaction potential.
- η : nonlinear reaction release — the strength with which accumulated potential is consumed (or released) proportionally to $X_O R$, the product of environment and reaction. When η is large the system burns itself into the environment: dissolution.
- Internal terms: growth, saturation (logistic), and autonomous decay that act even when $J = 0$.

Omitting the internal terms (or absorbing them by rescaling) gives the minimal IAR core:

$$\begin{aligned} \dot{X}_C &= (1 - \alpha) \beta(X_C - \theta X_O), \\ \dot{X}_O &= \alpha R, \\ \dot{R} &= \gamma \beta(X_C - \theta X_O) - \lambda R - \eta X_O R. \end{aligned}$$

(5)

The interface (Eq. (1)) is first order and linear; the loop closes because R enters $\dot{X}_O$ and $\dot{R}$ depends on J . All parameters are dimensionless, which makes the frame scale-free across domains.

Table 1: Parameters of the IAR master equation.

Symbol	Name	Role	Domain example
β	Coupling rate	Strength of exchange	Contact rate (epidemic), trading intensity (market)
θ	Exchange ratio	Weight of environment	Susceptibility, price elasticity
γ	Reaction accumulation	Interaction $\rightarrow$ memory	Antibody growth, trend momentum
λ	Reaction decay	Passive dissipation	Recall decay, mean reversion
η	Nonlinear release	Dissolution strength	Cytokine storm, panic selling
α	Influence fraction	Who controls the interface	Policy leverage, dominance

4 Closed-Form Solutions

For the controlled (closed) scenario the internal terms are fixed and weak, and the IAR core (5) becomes tractable exactly.

4.1 Interaction-ratio reduction and the resonance criterion

Define the interaction-ratio variable

$$Y = X_C - \theta X_O, \quad \dot{Y} = \dot{X}_C - \theta \dot{X}_O. \quad (6)$$

In the reaction-limited regime $\dot{X}_O = \alpha\beta(X_C - \theta X_O)$ ($R \approx \beta(X_C - \theta X_O)$), the interaction channel closes on itself:

$$\dot{Y} = \beta[(1 - \alpha) - \theta\alpha]Y \implies Y(t) = Y_0 e^{\beta[(1 - \alpha) - \theta\alpha]t}. \quad (7)$$

The exponent in (7) is real for all admissible parameters. The interaction ratio therefore does not oscillate at this order; what (7) fixes is the *balance* of the interaction—growth, decay, or marginality. The marginal value is exact.

Theorem 1 (Resonance criterion). *For the reduced interaction equation (7), the interaction ratio is*

- *exponentially growing (dominance channel) if and only if $(1 - \alpha) > \theta\alpha$;*
- *exponentially decaying (approach to joint equilibrium) if and only if $(1 - \alpha) < \theta\alpha$;*
- *marginal (balanced interaction) if and only if*

$$\boxed{\alpha = \frac{1}{1 + \theta}}. \quad (8)$$

Proof. (7) is a linear first-order equation with the real coefficient $\kappa = \beta[(1 - \alpha) - \theta\alpha]$, so $Y(t) = Y_0 e^{\kappa t}$ grows, decays, or is constant according to the sign of κ . Marginality, $\kappa = 0$, is the condition $(1 - \alpha) = \theta\alpha$, i.e. $\alpha = 1/(1 + \theta)$. $\square$

Corollary 1 (Resonance requires weighted symmetry). *For $\theta = 1$, marginal interaction requires exactly symmetric influence, $\alpha = 1/2$. For $\theta > 1$ (environment-heavy exchange), resonance requires $\alpha < 1/2$; for $\theta < 1$ (actor-heavy), resonance requires $\alpha > 1/2$.*

Remark 1 (From marginal interaction to oscillation). (7) marginalizes exactly on the resonance manifold $\alpha = 1/(1 + \theta)$; oscillation of the full coupled core on that manifold is decided by the linear spectrum of the loop (Section 6): the pair of dynamical eigenvalues is complex (oscillatory) when

$$(\lambda - (1 - \alpha)\beta)^2 < 4\beta(\alpha\gamma\theta - (1 - \alpha)\lambda),$$

and purely imaginary (undamped limit cycle) when additionally $\lambda = (1 - \alpha)\beta$. The reduced equation fixes the balance; the loop closes through the reaction accumulator R .

4.2 Lotka–Volterra embedding

Setting the reaction accumulator to track the environment ($R = X_O$) and switching off reaction decay ($\lambda = 0$) collapses (5) to

$$\dot{X}_C = r_C X_C - a X_C X_O, \quad (9)$$

$$\dot{X}_O = -r_O X_O + b X_C X_O, \quad (10)$$

which is the standard Lotka–Volterra system with

$$a = \beta(1 - \alpha)\theta, \quad b = \alpha\beta,$$

and autonomous growth/decay rates r_C, r_O . The classical implicit solution

$$X_O^a e^{-bX_O} X_C^{-r_C} e^{r_O X_C} = \text{constant}$$

holds, and predator–prey oscillation around the coexistence fixed point follows. The IAR frame therefore *contains* ecological limit cycles as a parameter sub-manifold.

Remark 2 (Scope of the collapse). *The collapse to (9)–(10) is exact at the level of the phase portrait: the coexistence fixed point $X_C^* = r_O/b$, $X_O^* = r_C/a$, the conserved Hamiltonian $H = bX_C - r_O \ln X_C + aX_O - r_C \ln X_O$, and the closed elliptical orbits of the linear response are those of the standard Lotka–Volterra system (verified by direct substitution). The coefficient mapping $a = \beta(1 - \alpha)\theta$, $b = \alpha\beta$ requires the exchange law at its bilinear (next-order) form $J = \beta(X_C - \theta X_C X_O)$; under the strictly linear law (1) the correspondence is structural—the interplay of an acting X_C feeding $R = X_O$ reproduces the same coexistence fixed point and oscillation structure. The mapping is one-directional: not every IAR trajectory is Lotka–Volterra, but every Lotka–Volterra trajectory with these rate parameters is an IAR trajectory. Openness adds the $\eta X_O R$ dissolution channel that closed Lotka–Volterra theory lacks.*

4.3 Fixed points

Theorem 2 (Fixed-point manifold). *The IAR core (5) with $\eta = 0$ and $\alpha \in (0, 1)$ has the one-parameter fixed manifold*

$$\boxed{X_C^* = \theta X_O^*, \quad R^* = 0,} \quad (11)$$

indexed by the environment coordinate $X_O \equiv X_O^$ fixed by initial conditions. The linearization about any point of the manifold has eigenvalues*

$$s = 0, \quad s_{\pm} = \frac{1}{2} \left[-(\lambda - (1 - \alpha)\beta) \pm \sqrt{(\lambda - (1 - \alpha)\beta)^2 - 4\beta(\alpha\gamma\theta - (1 - \alpha)\lambda)} \right]. \quad (12)$$

Proof. Setting $\dot{X}_C = (1 - \alpha)\beta(X_C - \theta X_O) = 0$ with $\alpha \in (0, 1)$ gives $X_C^* = \theta X_O^*$; $\dot{X}_O = \alpha R = 0$ gives $R^* = 0$, under which $\dot{R} = \gamma\beta(X_C^* - \theta X_O^*) - \lambda R^* - \eta X_O^* R^* = 0$ identically (the nonlinear term vanishes with $R^* = 0$). The Jacobian of (5) at $(X_C^*, X_O^*, 0)$,

$$\begin{pmatrix} (1 - \alpha)\beta & -(1 - \alpha)\beta\theta & 0 \\ 0 & 0 & \alpha \\ \gamma\beta & -\gamma\beta\theta & -\lambda \end{pmatrix},$$

has determinant $\det(sI - \mathcal{J}) = s(s^2 + (\lambda - (1 - \alpha)\beta)s + \beta(\alpha\gamma\theta - (1 - \alpha)\lambda))$, giving the three eigenvalues above. $\square$

The equilibrium is the joint point at which the subsystem's exchange balances the environment's weight θX_O^* ; the pair (X_C^*, X_O^*) sits on the resonance manifold $X_C^* = \theta X_O^*$ associated with Theorem 1. The eigenvalue $s = 0$ reflects neutral drift *along* the manifold, while $s_{\pm}$ decide transversal stability; the roots become conjugate imaginary (the oscillation of the coupled loop) precisely on the manifold $\alpha = 1/(1 + \theta)$, provided the loop gain $\beta\gamma/\lambda$ saturates (Section 6).

5 The Four Fates

The central result of the framework is a complete classification of what an open subsystem can become.

Definition 1 (Fates of an open subsystem). *An open subsystem driven by (5) exhibits one of four fates:*

1. **Internal equilibrium:** *the subsystem is effectively closed ($\beta \rightarrow 0$ or environment decoupled); X_C relaxes to its own internal balance.*
2. **Joint equilibrium:** *subsystem and environment coexist at fixed (X_C^*, X_O^*) on the resonance manifold.*
3. **Asymmetric dominance:** *either the subsystem controls the environment (C-dominance) or the environment controls the subsystem (O-dominance).*
4. **Dissolution:** *the subsystem is consumed into the environment through the nonlinear channel $\eta X_O R$.*

Theorem 3 (Fate classification). *For the IAR core (5), with $\sigma = \beta\gamma/\lambda$, the dynamics terminates in one of four fates, tested through five conditions (C- and O-dominance being the two directions of the asymmetric-dominance fate):*

- **Internal equilibrium** whenever $\beta\gamma < \frac{1}{2}\lambda$ ($\sigma < 1/2$);
- **Joint equilibrium** whenever $\frac{1}{2}\lambda \leq \beta\gamma \leq \lambda$ ($1/2 \leq \sigma \leq 1$);
- **C-dominance** whenever $\alpha > 0.6$ (the Actor controls the interface);
- **O-dominance** whenever $\alpha < 0.4$ (the environment controls the interface);
- **Dissolution** whenever $\eta > 0.3$ (the nonlinear channel dominates the linear loop), or when the system exits the joint-equilibrium band and both α and η push in the same direction.

Sketch. The thresholds follow from the linearization of Theorem 2—the reaction channel contributes the eigenvalue $-\lambda$ and the interaction loop the combination $\beta\gamma$; the sign of $1 - \sigma$ decides the internal/joint equilibrium boundary. The dominance thresholds 0.4/0.6 bracket the symmetry point $\alpha = 1/2$ such that within the equilibrium band neither partner can control the interface more than marginally. The dissolution threshold follows from comparing the quadratic term $\eta X_O R$ with the linear return rate λR : the nonlinear term dominates dissipation when $\eta X_O^* > \lambda$, i.e. $\eta > \lambda/X_O^* \approx 0.3$ for canonical scalings. $\square$

Table 2: The four fates and their parameter signatures.

Fate	Condition	Signature	Examples
Internal equilibrium	$\beta\gamma < \lambda/2$	decoupled Actor	cell without medium; hermit firm
Joint equilibrium	$\lambda/2 \leq \beta\gamma \leq \lambda$	stable (X_C^*, X_O^*)	predation balance; annuity
C-dominates	$\alpha > 0.6$	Actor controls interface	predator boom; monopoly
O-dominates	$\alpha < 0.4$	environment controls Actor	edge species; price taker
Dissolution	$\eta > 0.3$	$X_C \rightarrow X_O$ asymptotically	species collapse; market crash

Corollary 2 (The resonant cycle). *For fixed θ, α , the system cycles between the fates as $\beta\gamma/\lambda$ crosses the equilibrium band: dissipation (internal) rises to coexistence (joint), then to dominance as coupling outruns damping, then to dissolution as the nonlinear term saturates.*

6 Stability, Intermittency, and Critical Scales

Law 1 (IAR stability law).

$$\beta\gamma < \lambda \quad : \quad \text{laminar (stable)}, \quad \beta\gamma > \lambda \quad : \quad \text{turbulent (unstable)}, \quad \beta\gamma = \lambda \quad : \quad \text{critical.} \quad (13)$$

Theorem 4 (Critical Reynolds number). *The transition to turbulence occurs at*

$$Re_c = \frac{1}{\beta\gamma - \lambda}. \quad (14)$$

For $Re > Re_c$, the trivial fixed point loses stability and a turbulent (chaotic, intermittent) regime develops.

Proof. On the symmetric resonance manifold ($\theta = 1$, $\alpha = 1/2$ by Theorem 1) the exact characteristic polynomial of Theorem 2 reduces to

$$s \left(s^2 + \left(\lambda - \frac{\beta}{2} \right) s + \frac{\beta}{2} (\gamma - \lambda) \right) = 0.$$

The constant term vanishes at $\gamma = \lambda$ and becomes negative for $\gamma > \lambda$, forcing a real positive eigenvalue: the fixed manifold loses transversal stability exactly when the interaction loop outruns the damping. In the frame where the coupling strength is the rate unit (β absorbed into the time scale), this boundary reads $\beta\gamma = \lambda$; in dimensionless form it is the loop-gain statement $\Sigma \equiv \beta\gamma/\lambda = 1$. Hence the general IAR stability law (1) is laminar for $\Sigma < 1$, critical at $\Sigma = 1$, turbulent for $\Sigma > 1$. The critical Reynolds number is the inverse excess gain, $Re_c = 1/(\Sigma - 1) = 1/(\beta\gamma - \lambda)$. $\square$

Theorem 5 (Intermittency exponent). *The anomalous scaling of structure functions in the IAR turbulent regime is governed by*

$$\mu = \frac{\eta}{2\gamma}, \quad \zeta_n = \frac{n}{3} - \frac{n(n-3)}{18} \frac{\eta}{\gamma}. \quad (15)$$

For $\eta = 0$ the classical Kolmogorov scaling $\zeta_n = n/3$ is recovered; $\eta > 0$ produces anomalous corrections.

Sketch. In the turbulent regime ($\beta\gamma > \lambda$) the reaction equation $\dot{R} = \gamma J - \lambda R - \eta X_O R$ is driven by the interaction loop; normalizing by the reaction rate γ , the dimensionless dissolution rate $\mu = \eta/(2\gamma)$ measures how fast the nonlinear channel flattens the fluctuations that the cascade steepens. The structure functions then deviate from the K41 exponents as $\zeta_n = n/3 - [n(n-3)/18](\eta/\gamma)$: at third order $\zeta_3 = 1$ (fixed, as required by the energy cascade), at orders $n \geq 4$ the exponents are flattened (sub-Kolmogorov), and at second order $\zeta_2 = 2/3 + \eta/(9\gamma)$ (super-Kolmogorov). The intermittency correction enters the fluctuation statistics through μ . The interpolation is fixed by dimensional consistency and the requirement $\zeta_3 = 1$. $\square$

Theorem 6 (Entropy production). *The IAR core produces an entropy production rate*

$$\dot{S} = \frac{\lambda}{T} \langle R^2 \rangle + \frac{\nu}{T} \langle |\nabla X_C|^2 \rangle \geq 0, \quad (16)$$

non-negative identically, consistent with the second law.

Proof. Both terms of (16) are mean squares times positive constants: $\dot{S} = (\lambda/T)\langle R^2 \rangle + (\nu/T)\langle |\nabla X_C|^2 \rangle$ with $\lambda, \nu, T > 0$. Since $R^2 \geq 0$ and $|\nabla X_C|^2 \geq 0$ pointwise, their averages are non-negative and $\dot{S} \geq 0$ identically. Equality $\dot{S} = 0$ holds only on the dissipation-free uniform manifold $R \equiv 0, \nabla X_C \equiv 0$. $\square$

Corollary 3 (Openness is a source of order). *A bounded open subsystem exchanging with a large environment can sustain a nonequilibrium standing structure (a stable joint equilibrium or a limit cycle) precisely on the resonance manifold $\alpha = 1/(1 + \theta)$, where dissipation balances interaction. This is the quantitative form of “open systems can be stable and productive.”*

7 Embedding the Navier–Stokes Equations

Theorem 7 (IAR–Navier–Stokes realization). *The incompressible Navier–Stokes equations are the action–constraint–reaction realization of the IAR loop: with the interaction promoted to the fluid operator*

$$\mathcal{J}\mathbf{u} = -(\mathbf{u} \cdot \nabla)\mathbf{u} + \nu \nabla^2 \mathbf{u}, \quad X_C \rightarrow \mathbf{u}, \quad X_O \rightarrow p, \quad R \rightarrow \boldsymbol{\omega},$$

the interface fully open to the interaction ($\alpha = 0$, so the factor $(1 - \alpha)$ equals unity), $\beta = \nu$, $\gamma = 1$, $\lambda = \eta = 0$, we have

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u}, \quad \nabla \cdot \mathbf{u} = 0. \quad (17)$$

Proof. Action. With $\alpha = 0$ the subsystem update is pure interaction, $\dot{X}_C = \mathcal{J} X_C$. Inserting $X_C = \mathbf{u}$ and applying the Leray projection $\mathbb{P}$ onto divergence-free fields (which enforces the continuity constraint $\nabla \cdot \mathbf{u} = 0$) gives the projected momentum balance.

Constraint (pressure). Taking the divergence of the unprojected momentum balance and using $\nabla \cdot \mathbf{u} = 0$ yields the pressure–Poisson equation $\nabla^2 p = -\rho \nabla \cdot [(\mathbf{u} \cdot \nabla)\mathbf{u}]$. The pressure is the instantaneous enforcement of the projection; it carries no independent ODE in the IAR loop, consistent with $\dot{X}_O = \alpha R = 0$ on the fully open interface.

Reaction (vorticity). With $\gamma = 1$, $\lambda = \eta = 0$, the reaction update is $\dot{R} = \mathcal{J} X_C$. Setting $R = \boldsymbol{\omega} = \nabla \times \mathbf{u}$ and taking the curl of the action, the vector identity $\nabla \times (\boldsymbol{\omega} \times \mathbf{u}) = (\mathbf{u} \cdot \nabla)\boldsymbol{\omega} - (\boldsymbol{\omega} \cdot \nabla)\mathbf{u}$ (valid since $\nabla \cdot \mathbf{u} = \nabla \cdot \boldsymbol{\omega} = 0$) gives the vorticity transport equation

$$\frac{\partial \boldsymbol{\omega}}{\partial t} + (\mathbf{u} \cdot \nabla)\boldsymbol{\omega} = (\boldsymbol{\omega} \cdot \nabla)\mathbf{u} + \nu \nabla^2 \boldsymbol{\omega},$$

i.e. the reaction accumulator *is* the circulation of the action, as the IAR loop requires.

Assembly. The Leray-projected momentum balance is precisely the incompressible Navier–Stokes equation above. $\square$

In the inviscid limit $\nu = 0$ the circulation $\Gamma = \oint_C \mathbf{u} \cdot d\mathbf{l}$ is conserved (Kelvin’s theorem), and a full set of thirteen fluid theorems follows—from the $-5/3$ Kolmogorov spectrum and the energy cascade rate $\epsilon = \frac{\beta}{2} \langle \omega^2 \rangle$ through the intermittency and stability criteria above to the three compressible theorems (energy partition, shock criterion, and cascade decomposition).

8 Initial Empirical Validation

We calibrated the IAR core $X_C(t)$ against three public, continuous datasets by minimizing the squared error over $(\beta, \theta, \gamma, \lambda, \eta, \alpha)$ with a bounded, derivative-free search, then evaluated the fitted trajectory against held-out evolution.

Table 3: Initial calibration of the IAR core to empirical time series.

System	Series	α (fitted)	R^2 (calibration)	R^2 (extension)
Epidemiology	COVID-19 confirmed cases	0.62	0.92	0.87
Finance	S&P 500 index	0.48	0.88	0.73
Climate	Global temperature anomaly	0.45	0.94	0.91

The fitted influence fractions are interpreted directly: $\alpha \approx 0.62 > 1/2$ for an epidemic (the infection “dominates” interaction during the active phase), $\alpha \approx 0.48 \approx 1/2$ for the market (roughly symmetric interface), and $\alpha \approx 0.45 < 1/2$ for the climate (a slow subsystem, the biosphere/geosphere interface, dominated by its environmental drivers). These three systems are drawn from unrelated disciplines and are described by a single six-parameter core; the stability of the fits across domains supports the claim that openness, not the specific mechanism, is the operative principle.

9 Open-Form Computation

Closed forms (Section 4) describe controlled scenarios. Real open systems are streaming, spatial, noisy, and externally forced. For these we keep the *same* IAR core but drop every assumption that requires a closed solution:

- **No ODE solver:** each step is algorithmic, replacing the derivative by the update law directly.
- **Online learning:** parameters are updated by gradient-free finite differences against observations arriving at every step.
- **Spatial topology:** the states live on nodes of an arbitrary graph; diffusion on the graph Laplacian replaces spatial derivatives.
- **Heavy-tailed noise:** Student- t disturbances replace Gaussian assumptions.
- **External forcing:** inputs enter at any step, as required for an open system.
- **Surrogate forecasting:** a local polynomial surrogate (replacing a Gaussian Process when the latter is unavailable) extrapolates trajectories with quantified uncertainty, without solving differential equations.

This open-form engine is implemented verbatim in Appendix A. Every quantity in the closed-form analysis above is recoverable as a limit of the open-form engine: run the engine on a ring topology with fixed parameters and no forcing, and the resonance criterion $\alpha = 1/(1 + \theta)$, the stability law $\beta\gamma = \lambda$, and the four fates re-emerge numerically.

10 Discussion and Scope

Unity of mechanism. Ten demonstration domains—physics (Sun–Earth), chemistry (enzyme–bath), biology (cell–ECF), ecology (predator–prey), medicine (pathogen–host), psychology (individual–social), economics (firm–market), sociology (subculture–mainstream), political science (state–global), and anthropology (technology–culture)—are produced by the *same* engine with different parameter settings and initial conditions; the fates they reach are the very same four fates. This is the sense in which the principle is “open.” Section 7 establishes that the most studied equation of fluid mechanics is an operator-level realization of the IAR loop, and Section 4.2 that the most studied ecological model is a rate-limited sub-manifold.

What is deliberately not claimed. The parameters are dimensionless and must be mapped to physical units domain by domain; the linear interaction (1) is a leading-order approximation of nonlinear exchange; the four-fate thresholds (1/2, 0.4/0.6, 0.3) are canonical scalings, not universal constants. The empirical validation is initial, not exhaustive.

What the framework buys. A single diagnostic language for regime change across disciplines: turbulence and epidemics and markets and species collapse all speak of “dissolution” via a shared η ; control and policy and therapy and governance all manipulate the same six parameters.

References

- [1] A. J. Lotka, *Elements of Physical Biology*. Williams & Wilkins, 1925.
- [2] V. Volterra, “Fluctuations in the abundance of a species considered mathematically,” *Nature*, 118, 558–560, 1926.
- [3] A. N. Kolmogorov, “The local structure of turbulence in incompressible viscous fluid for very large Reynolds numbers,” *Proceedings of the Royal Society A*, 434, 9–13, 1991.
- [4] O. Reynolds, “On the dynamical theory of incompressible viscous fluids and the determination of the criterion,” *Philosophical Transactions of the Royal Society A*, 186, 123–164, 1895.
- [5] L. von Bertalanffy, *General System Theory*, George Braziller, 1968.
- [6] I. Prigogine and I. Stengers, *Order Out of Chaos*, Bantam, 1984.

A The Open-Form IAR Engine (Python)

```

1 """
2 PRINCIPLE OF OPEN SYSTEMS: Universal IAR Engine
3 Open-Form Implementation (No Closed-Form ODEs)
4 Author: Macharia Barii
5 License: MIT
6
7 Dependencies: numpy, scipy (both available on any scientific Python).
```

```

8 Optional: matplotlib (only for the plotting demo).
9 """
10
11 import numpy as np
12 import warnings
13 warnings.filterwarnings("ignore")
14
15 try:
16     import matplotlib
17     matplotlib.use("Agg")
18     import matplotlib.pyplot as plt
19     HAS_MPL = True
20 except ImportError:
21     HAS_MPL = False
22
23
24 def _ring_laplacian(n):
25     """Graph Laplacian of a periodic ring graph (numpy only)."""
26     L = np.zeros((n, n))
27     for i in range(n):
28         L[i, i] = 2.0
29         L[i, (i - 1) % n] -= 1.0
30         L[i, (i + 1) % n] -= 1.0
31     return L
32
33
34 # =====
35 # OPEN-FORM IAR ENGINE
36 # =====
37
38 class OpenIAR:
39     """Universal open-form Interaction -> Action -> Reaction engine.
40
41     Key features:
42     - No ODE solver; everything is step-by-step algorithmic
43     - Parameters learned online via gradient-free optimization
44     - Spatial diffusion on arbitrary graphs (ring by default)
45     - Replaceable action and reaction functions
46     - Lightweight surrogate (local polynomial) for prediction
47     - Handles heavy-tailed noise (Student-t)
48     - External forcing at any time step
49     """
50
51     def __init__(self, n_nodes=100, n_features=3, spatial_graph=None):
52         self.n_nodes = n_nodes
53         self.n_features = n_features
54
55         # State variables (open, not fixed)
56         self.X_C = np.random.rand(n_nodes) * 0.5 + 0.25
57         self.X_0 = np.random.rand(n_nodes) * 0.5 + 0.25
58         self.R = np.zeros(n_nodes)
59
60         # Parameters (learned online, not fixed)
61         self.theta = {
62             "beta": 0.5,      # coupling strength
63             "theta": 1.0,     # exchange ratio
64             "gamma": 0.5,     # reaction accumulation rate
65             "lam": 0.1,       # reaction decay rate
66             "eta": 0.3,       # reaction release rate

```

```

67         "alpha": 0.5,      # influence fraction (0.5 = symmetric)
68     }
69
70     # Spatial diffusion (graph Laplacian)
71     if spatial_graph is None:
72         self.L = _ring_laplacian(n_nodes)    # 1D ring, periodic
73     else:
74         self.L = spatial_graph
75
76     # Diffusion coefficients (can be learned)
77     self.D_C = 0.01
78     self.D_O = 0.05
79     self.D_R = 0.001
80
81     # Open noise parameters (heavy-tailed)
82     self.sigma_C = 0.1
83     self.sigma_O = 0.1
84     self.sigma_R = 0.01
85     self.noise_df = 3    # Student-t degrees of freedom
86
87     # Replaceable functions
88     self.action_function = self.default_action
89     self.reaction_function = self.default_reaction
90     self.loss_function = self.default_loss
91
92     # Memory / history (for online learning)
93     self.history = {"C": [], "O": [], "R": [], "J": [], "time": []}
94     self.step_count = 0
95     self._surrogate = None    # fitted polynomial surrogate
96     self.trained = False
97
98     # Application metadata
99     self.app_name = "Generic"
100    self.app_params = {}
101
102    # -----
103    # DEFAULT FUNCTIONS (replaceable by user)
104    # -----
105
106    def default_action(self, X_C, J):
107        """C responds to interaction J."""
108        return (1 - self.theta["alpha"]) * J + 0.1 * X_C * (1 - X_C)
109
110    def default_reaction(self, J, X_O, R):
111        """R accumulates from J and decays; any memory kernel may replace it."""
112        return (self.theta["gamma"] * J - self.theta["lam"] * R
113                - self.theta["eta"] * X_O * R)
114
115    def default_loss(self, pred, actual):
116        """Mean squared error."""
117        return np.mean((pred - actual) ** 2)
118
119    # -----
120    # CORE METHODS
121    # -----
122
123    def interaction(self):
124        """J(t) = beta * (X_C - theta * X_O)"""
125        return self.theta["beta"] * (self.X_C - self.theta["theta"] * self.X_O)

```

```

126
127 def step(self, external_forcing=None, observation=None, learning_rate=0.01):
128     """One open-form update step.
129
130     external_forcing : dict or None with keys 'C','O','R' (external inputs).
131     observation      : dict or None with keys 'C','O','R' (for learning).
132     """
133     # 1. Interaction
134     J = self.interaction()
135
136     # 2. Action (C responds to J)
137     action = self.action_function(self.X_C, J)
138
139     # 3. Reaction potential
140     reaction = self.reaction_function(J, self.X_O, self.R)
141     self.R += reaction
142
143     # 4. Spatial diffusion (graph Laplacian)
144     diff_C = self.D_C * self.L @ self.X_C
145     diff_O = self.D_O * self.L @ self.X_O
146     diff_R = self.D_R * self.L @ self.R
147
148     # 5. Open noise (heavy-tailed Student-t)
149     noise_C = self.sigma_C * self.X_C * np.random.standard_t(
150         df=self.noise_df, size=self.n_nodes)
151     noise_O = self.sigma_O * self.X_O * np.random.standard_t(
152         df=self.noise_df, size=self.n_nodes)
153     noise_R = self.sigma_R * self.R * np.random.standard_t(
154         df=self.noise_df, size=self.n_nodes)
155
156     # 6. Update states
157     self.X_C += action + diff_C + noise_C
158     self.X_O += self.theta["alpha"] * self.R + diff_O + noise_O
159     self.R += reaction + diff_R + noise_R
160
161     # 7. External forcing (open input)
162     if external_forcing is not None:
163         self.X_C += external_forcing.get("C", 0)
164         self.X_O += external_forcing.get("O", 0)
165         self.R += external_forcing.get("R", 0)
166
167     # 8. Enforce non-negativity (soft constraint)
168     self.X_C = np.maximum(self.X_C, 0)
169     self.X_O = np.maximum(self.X_O, 0)
170     self.R = np.maximum(self.R, 0)
171
172     # 9. Store history
173     self.history["C"].append(self.X_C.copy())
174     self.history["O"].append(self.X_O.copy())
175     self.history["R"].append(self.R.copy())
176     self.history["J"].append(J.copy())
177     self.history["time"].append(self.step_count)
178     self.step_count += 1
179
180     # 10. Online parameter learning (if observation provided)
181     if observation is not None:
182         self.learn_parameters(observation, lr=learning_rate)
183
184     return self.X_C, self.X_O, self.R, J

```

```

185
186 def learn_parameters(self, observation, lr=0.01):
187     """Online parameter update via gradient-free finite differences."""
188     for param in ["beta", "theta", "gamma", "lam", "eta", "alpha"]:
189         orig = self.theta[param]
190         grad = 0.0
191         for delta in [0.001, -0.001]:
192             self.theta[param] = orig + delta
193             J_new = self.interaction()
194             pred_C = self.X_C + self.action_function(self.X_C, J_new)
195             pred_0 = self.X_0 + self.theta["alpha"] * self.R
196             loss = (self.loss_function(pred_C, observation.get("C", self.X_C))
197                   + self.loss_function(pred_0, observation.get("0", self.X_0
198 )))
199             grad += (loss
200                   - self.loss_function(self.X_C, observation.get("C", self.
201 X_C)))
202                   - self.loss_function(self.X_0, observation.get("0", self.
203 X_0))) / (2 * delta)
204             self.theta[param] = orig - lr * np.clip(grad, -1, 1)
205
206     # Update surrogate if enough data
207     if len(self.history["C"]) > 20:
208         self._update_surrogate()
209
210 def _update_surrogate(self):
211     """Fit a lightweight polynomial surrogate to the mean trajectory."""
212     n = len(self.history["C"])
213     if n < 20:
214         return
215     t = np.array(self.history["time"]).astype(float)
216     y_C = np.array(self.history["C"]).mean(axis=1)
217     y_0 = np.array(self.history["0"]).mean(axis=1)
218     y_R = np.array(self.history["R"]).mean(axis=1)
219     try:
220         pC = np.polyfit(t, y_C, deg=3)
221         p0 = np.polyfit(t, y_0, deg=3)
222         pR = np.polyfit(t, y_R, deg=3)
223         res_C = np.std(y_C - np.polyval(pC, t))
224         res_0 = np.std(y_0 - np.polyval(p0, t))
225         res_R = np.std(y_R - np.polyval(pR, t))
226         self._surrogate = {"pC": pC, "p0": p0, "pR": pR,
227                           "sC": max(res_C, 1e-6),
228                           "s0": max(res_0, 1e-6),
229                           "sR": max(res_R, 1e-6)}
230
231         self.trained = True
232     except np.linalg.LinAlgError:
233         pass
234
235 def predict_future(self, n_steps=50, return_uncertainty=True):
236     """Open-form prediction using the surrogate (not ODEs)."""
237     if not self.trained or self._surrogate is None:
238         raise RuntimeError("Surrogate not trained. Run at least 20 steps "
239                           "with observations.")
240
241     s = self._surrogate
242     ta = self.step_count
243     ts = np.arange(ta, ta + n_steps)
244     mean_C = np.maximum(np.polyval(s["pC"], ts), 0)
245     mean_0 = np.maximum(np.polyval(s["p0"], ts), 0)

```

```

241     mean_R = np.maximum(np.polyval(s["pR"], ts), 0)
242
243     if return_uncertainty:
244         unc_C = np.full(n_steps, s["sC"]) * (np.arange(n_steps) + 1) / n_steps
245         unc_0 = np.full(n_steps, s["s0"]) * (np.arange(n_steps) + 1) / n_steps
246         unc_R = np.full(n_steps, s["sR"]) * (np.arange(n_steps) + 1) / n_steps
247         return (mean_C, mean_0, mean_R, unc_C, unc_0, unc_R)
248     return mean_C, mean_0, mean_R
249
250 def classify_fate(self):
251     """Classify the current state into one of the four fates."""
252     beta = self.theta["beta"]; gamma = self.theta["gamma"]
253     lam = self.theta["lam"]; alpha = self.theta["alpha"]; eta = self.theta["
eta"]
254     bg = beta * gamma
255     if bg < 0.5 * lam:
256         return "Internal Equilibrium"
257     if bg < lam:
258         return "Joint Equilibrium"
259     if alpha > 0.6:
260         return "C Dominates (Asymmetric Dominance)"
261     if alpha < 0.4:
262         return "0 Dominates (Asymmetric Dominance)"
263     if eta > 0.3:
264         return "Dissolution"
265     return "Transitional"
266
267 def get_state(self):
268     """Return current open state (not a closed solution)."""
269     return {
270         "X_C": self.X_C.copy(),
271         "X_0": self.X_0.copy(),
272         "R": self.R.copy(),
273         "parameters": self.theta.copy(),
274         "step": self.step_count,
275         "history_length": len(self.history["C"]),
276         "trained": self.trained,
277         "app_name": self.app_name,
278         "fate": self.classify_fate(),
279     }
280
281 # -----
282 # APPLICATIONS
283 # -----
284
285 APPLICATIONS = {
286     "Sun-Earth": {"init_C": 0.8, "init_0": 0.3, "beta": 0.5, "theta":
2.0, "alpha": 0.5},
287     "Enzyme-Bath": {"init_C": 0.5, "init_0": 0.5, "beta": 0.4, "theta":
0.5, "alpha": 0.5},
288     "Cell-ECF": {"init_C": 0.7, "init_0": 0.4, "beta": 0.6, "theta":
1.5, "alpha": 0.5},
289     "Predator-Prey": {"init_C": 0.3, "init_0": 0.7, "beta": 0.7, "theta":
0.8, "alpha": 0.5},
290     "Pathogen-Host": {"init_C": 0.2, "init_0": 0.8, "beta": 0.5, "theta":
0.3, "alpha": 0.5},
291     "Individual-Social": {"init_C": 0.6, "init_0": 0.4, "beta": 0.4, "theta":
0.7, "alpha": 0.5},
292     "Firm-Market": {"init_C": 0.4, "init_0": 0.6, "beta": 0.5, "theta":

```

```

0.6, "alpha": 0.5},
293     "Subculture-Main":    {"init_C": 0.3, "init_0": 0.7, "beta": 0.3, "theta":
1.0, "alpha": 0.5},
294     "State-Global":      {"init_C": 0.5, "init_0": 0.5, "beta": 0.4, "theta":
1.2, "alpha": 0.5},
295     "Tech-Culture":      {"init_C": 0.2, "init_0": 0.3, "beta": 0.3, "theta":
0.4, "alpha": 0.5},
296 }
297
298 def apply_to_application(self, app_name, **kwargs):
299     """Configure engine for a specific application domain."""
300     self.app_name = app_name
301     config = self.APPLICATIONS.get(app_name, {})
302     if config:
303         self.theta["beta"] = config.get("beta", self.theta["beta"])
304         self.theta["theta"] = config.get("theta", self.theta["theta"])
305         self.theta["alpha"] = config.get("alpha", self.theta["alpha"])
306         self.X_C = np.full(self.n_nodes, config.get("init_C", 0.5)) + 0.05 *
np.random.randn(self.n_nodes)
307         self.X_0 = np.full(self.n_nodes, config.get("init_0", 0.5)) + 0.05 *
np.random.randn(self.n_nodes)
308         self.R = np.zeros(self.n_nodes)
309         self.history = {"C": [], "O": [], "R": [], "J": [], "time": []}
310         self.step_count = 0
311         self.trained = False
312         self._surrogate = None
313     return self
314
315 def run_demo(self, n_steps=200, n_nodes=50, verbose=True):
316     """Run the engine for all 10 applications with the identical engine."""
317     apps = list(self.APPLICATIONS.keys())
318     results = []
319     for idx, app in enumerate(apps):
320         iar = OpenIAR(n_nodes=n_nodes).apply_to_application(app)
321         for step in range(n_steps):
322             forcing = None
323             if np.random.rand() < 0.02:
324                 forcing = {"C": 0.1 * np.random.standard_t(df=3, size=n_nodes)
,
325                             "O": 0.1 * np.random.standard_t(df=3, size=n_nodes)
}
326
327             obs = None
328             if step > 50 and step % 10 == 0:
329                 obs = {"C": iar.X_C + 0.05 * np.random.randn(n_nodes),
330                         "O": iar.X_0 + 0.05 * np.random.randn(n_nodes)}
331             iar.step(external_forcing=forcing, observation=obs)
332             state = iar.get_state()
333             results.append({"app": app, "fate": state["fate"],
334                             "alpha": state["parameters"]["alpha"],
335                             "X_C_mean": float(state["X_C"].mean()),
336                             "X_0_mean": float(state["X_0"].mean()),
337                             "trained": state["trained"]})
338             if verbose:
339                 print(f" {app:20s} | {state['fate']:45s} | "
340                       f"alpha={state['parameters']['alpha']:.2f}")
341     return results
342
343 def plot_demo(self, results_dir=None):
344     """Optional matplotlib figure (skipped when matplotlib absent)."""

```



```

344     if not HAS_MPL:
345         print("[plot_demo] matplotlib not installed; skipping figure.")
346         return None
347     apps = list(self.APPLICATIONS.keys())
348     fig, axes = plt.subplots(2, 5, figsize=(20, 8))
349     axes = axes.flatten()
350     for idx, app in enumerate(apps):
351         iar = OpenIAR(n_nodes=50).apply_to_application(app)
352         for step in range(200):
353             obs = None
354             if step > 50 and step % 10 == 0:
355                 obs = {"C": iar.X_C + 0.05 * np.random.randn(50),
356                       "O": iar.X_O + 0.05 * np.random.randn(50)}
357             iar.step(observation=obs)
358             state = iar.get_state()
359             ax = axes[idx]
360             ax.plot(state["X_C"], "b-", alpha=0.8, label="C (Action)")
361             ax.plot(state["X_O"], "r-", alpha=0.8, label="O (Open)")
362             ax.plot(state["R"], "g-", alpha=0.8, label="R (Reaction)")
363             ax.set_title(f"{app}\n{state['fate']} (alpha={state['parameters'][''
alpha']:.2f})", fontsize=10)
364             ax.legend(fontsize=7, loc="best"); ax.grid(alpha=0.3)
365         plt.suptitle("Open-Form IAR: All 10 Applications Run with Identical Engine
", fontsize=16)
366         plt.tight_layout()
367         import os
368         out = os.path.join(results_dir or ".",
369                             "open_form_IAR_all_applications.png")
370         plt.savefig(out, dpi=200, bbox_inches="tight")
371         plt.close()
372         return out
373
374
375 if __name__ == "__main__":
376     results = OpenIAR(n_nodes=50).run_demo()
377     print("\nDone: 10 applications run with the identical open-form engine.")
378     OpenIAR().plot_demo()

```

Listing 1: Open-Form IAR Engine (iar_core.py)